



Qnota

Sistema de Gestão Acadêmica

Uma plataforma integrada para gestão acadêmica e avaliação educacional

Equipe: Leonardo Mélo, Luís Felipe Perylo, Luís Felipe Soriano e Vinícius Beltrão

Outubro/2025



Contexto e Objetivos

A Qnota é voltada para instituições de ensino que buscam otimizar sua gestão acadêmica e avaliação do desempenho estudantil, apoiando tanto processos administrativos quanto pedagógicos.

Principais objetivos:



Organização de turmas

Gerenciamento eficiente de classes e distribuição de alunos



Controle de alunos e responsáveis

Dados cadastrais e relacionamento escola-família



Aplicação de simulados

Controle de avaliações com múltiplas disciplinas e pesos



Lançamento de notas

Registro e justificativas para alterações de notas



Geração de rankings

Classificação baseada na média ponderada das notas



Tomada de decisão pedagógica

Insights para professores e coordenação

Persona: Coordenador Pedagógico



O Coordenador Pedagógico é o principal usuário do sistema Qnota, responsável por gerenciar todo o processo educacional da instituição, desde a organização de turmas até a análise de desempenho dos alunos.

Características:



Gestor de processos acadêmicos

Responsável pela organização e controle das atividades educacionais



Líder pedagógico

Orienta professores e monitora progresso educacional dos alunos

Necessidades:



Transparência nas avaliações

Visualização clara dos dados e critérios de avaliação



Acompanhamento de resultados

Monitoramento da evolução acadêmica e identificação de padrões

Objetivos:



Otimização de processos

Aumentar a eficiência na gestão escolar



Melhorar qualidade educacional

Utilizar dados para aprimorar decisões pedagógicas

Jornada do Usuário

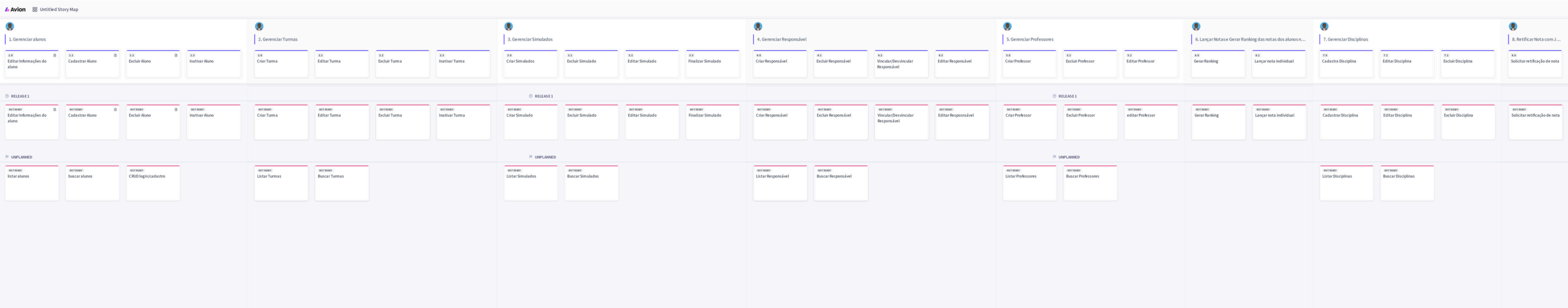
Principais passos da jornada do Coordenador:

-  1. Criação e organização de turmas
Estruturação das classes por ano letivo e distribuição de alunos
-  2. Matrícula e acompanhamento de alunos
Registro e monitoramento do histórico acadêmico dos estudantes
-  3. Aplicação e configuração de simulados
Definição de disciplinas, pesos e períodos de avaliação
-  4. Lançamento de notas e justificativas
Registro transparente das avaliações e suas alterações
-  5. Análise de rankings e evolução
Monitoramento do desempenho e tomada de decisões pedagógicas

Problemas identificados:

-  Falta de ferramentas integradas
Necessidade de utilizar múltiplos sistemas para gestão completa
-  Dificuldade de rastreio de desempenho
Ausência de ferramentas analíticas para acompanhamento da evolução
-  Processos manuais e burocráticos
Transferências, matrículas e lançamentos com pouca automação

Protótipo



[Storymap](#)

Acesse o protótipo completo no Figma



[Protótipo QNota Coordenador](#)

Explore a interface e fluxos do sistema em detalhes

Cenário de Teste: Transferência de Aluno

Cenário: Transferir aluno para turma do mesmo ano e sem simulados finalizados (sucesso - RN-57/RN-57a)

Given um "aluno" da turma "7A" "não possui" simulados finalizados

And a turma de destino "7B" pertence ao ano letivo "2025" igual ao da turma atual

When um coordenador transfere o "aluno" da turma "7A" para "7B"

Then o sistema confirma a transferência do "aluno"



Sobre o cenário de teste

Este cenário verifica as regras de negócio para transferência de alunos:

- RN-57: Não é possível transferir alunos com simulados finalizados
- RN-57a: Só é possível transferir para turmas do mesmo ano letivo

O teste valida que um aluno sem simulados finalizados pode ser transferido para uma turma do mesmo ano letivo com sucesso.

@Given: Aluno da turma possui ou não simulados finalizados

Primeiro passo do teste: configurar um aluno para verificar se possui ou não simulados finalizados

```
@Given("um \"aluno\" da turma {string} {string} simulados finalizados")
public void aluno_da_turma_possui_finalizados(String turmaOrigem, String possui) {
    currentTurmaId = ensureTurmaDefault(turmaOrigem);
    currentNome = "Aluno Teste";
    currentNascimento = LocalDate.of(2012, 1, 1);
    currentAlunoId = persistAlunoBasico(currentNome, currentNascimento, currentTurmaId,
                                       buildResponsaveisValidos(qtd:1, marcarUmPrincipal:true));

    if ("possui".equalsIgnoreCase(possui)) {
        var s = new Simulado(
            LocalDate.now().minusDays(daysToSubtract:10),
            Simulado.Status.FINALIZADO,
            currentTurmaId,
            List.of(dp(id:1, peso:6.0), dp(id:2, peso:4.0))
        );
        repo.salvar(s);
    }
}
```

</> Detalhes do método:

- Cria um aluno de teste na turma especificada
- Configura dados básicos do aluno (nome, data de nascimento, responsáveis)
- Verifica o parâmetro "possui" para determinar se deve criar um simulado finalizado para este aluno
- Se necessário, cria um simulado finalizado com duas disciplinas e pesos definidos

@Given: Turma de destino pertence ao ano letivo igual ao da turma atual

```
@Given("a turma de destino {string} pertence ao ano letivo {string} igual ao da turma atual")
public void turma_destino_ano_igual(String destinoAlias, String ano) {
    int origemAno = 2025;
    anoTurma.putIfAbsent(key:"7A", origemAno);
    ensureTurma(alias:"7A", origemAno);
    currentTurmaDestinoId = ensureTurma(destinoAlias, origemAno);
}
```



Detalhamento do método

Este método garante que a turma de destino esteja no mesmo ano letivo (2025) da turma de origem:

- Define o ano atual como 2025
- Verifica e registra o ano da turma de origem (7A)
- Configura a turma de destino (7B) com o mesmo ano letivo
- Armazena o ID da turma de destino para uso posterior no teste

Parte da verificação da regra de negócio RN-57a: "Só é possível transferir para turmas do mesmo ano letivo"



@When: Coordenador transfere o aluno de turma

```
@When("um coordenador transfere o \"aluno\" da turma {string} para {string}")
public void coord_transfere_aluno(String origemAlias, String destinoAlias) {
    lastError = null;
    try {
        if (currentTurmaDestinoId == null) currentTurmaDestinoId = ensureTurmaDefault(destinoAlias);
        alunoSrv.transferir(currentAlunoId, currentTurmaDestinoId);
    } catch (Exception e) { lastError = e; }
```



Sobre este método de teste

Este método implementa o passo "@When" do BDD que executa a ação principal do teste:

- Invoca o serviço de transferência de aluno entre turmas
- Trata possíveis exceções e armazena erros para validação posterior
- Utiliza o design pattern Try-Catch para capturar falhas durante a transferência

💡 Quando (@When): Define a ação principal que está sendo testada no cenário BDD

@Then: Sistema confirma a transferência do aluno

```
@Then("o sistema confirma a transferência do \"aluno\"")
public void confirma_transferencia() {
    assertNull(lastError, "Esperava sucesso na transferência: " + (lastError == null ? "" : lastError.getMessage()));
    var salvo = repo.findById(currentAlunoId).orElseThrow();
    assertEquals(currentTurmaDestinoId, salvo.getTurma(), message: "Turma não foi atualizada na persistência");
}
```



Verificação do resultado da transferência

Este método verifica se a transferência do aluno foi concluída com sucesso:

- Verifica se não houve erros durante o processo (`assertNull(lastError)`)
- Busca o aluno no repositório e confirma que a turma foi atualizada (`assertEquals`)
- Lança mensagem de erro caso a turma não tenha sido atualizada na persistência

💡 Este método faz parte da classe de testes automatizados que valida as regras de negócio RN-57/RN-57a

Implementação da Funcionalidade

```
// ----- TRANSFERIR -----  
public void transferir(AlunoId id, TurmaId novaTurma) {  
    var aluno = repo.porId(id).orElseThrow();  
  
    // RN-57.1: não pode se houver simulados finalizados  
    if (repo.possuiSimuladoFinalizado(id))  
        throw new IllegalStateException(s:"não é permitido alterar a turma do aluno com simulados finalizados");  
  
    // RN-57.2: mesma série/ano letivo  
    int anoAtual = turmaRepo.porId(aluno.getTurma()).map(Turma::getAnoLetivo).orElseThrow();  
    int anoNovo = turmaRepo.porId(novaTurma).map(Turma::getAnoLetivo).orElseThrow();  
    if (anoAtual != anoNovo)  
        throw new IllegalStateException(s:"a nova turma deve estar no mesmo ano letivo");  
  
    aluno.mudarTurma(novaTurma);  
    repo.salvar(aluno);  
}
```

Validação de Regras de Negócio

O método implementa duas regras importantes:

RN-57.1: Proibição de transferência quando há simulados finalizados

RN-57.2: Garantia que a nova turma pertença ao mesmo ano letivo

🔗 Fluxo de Execução

1. Busca do aluno pelo ID
2. Verificação das regras de negócio
3. Execução da transferência via `aluno.mudarTurma()`
4. Persistência da alteração com `repo.salvar()`

Obrigado